Exp No:8 FABRIC NETWORK DEPLOYMENT AND MANAGEMENT

AIM

To deploy, manage, monitor, and maintain a Hyperledger Fabric network by creating the network, verifying network components, deploying chaincode, monitoring Docker containers, and performing network shutdown and cleanup operations.

PROCEDURE

Step 1: Navigate to the Fabric Test Network

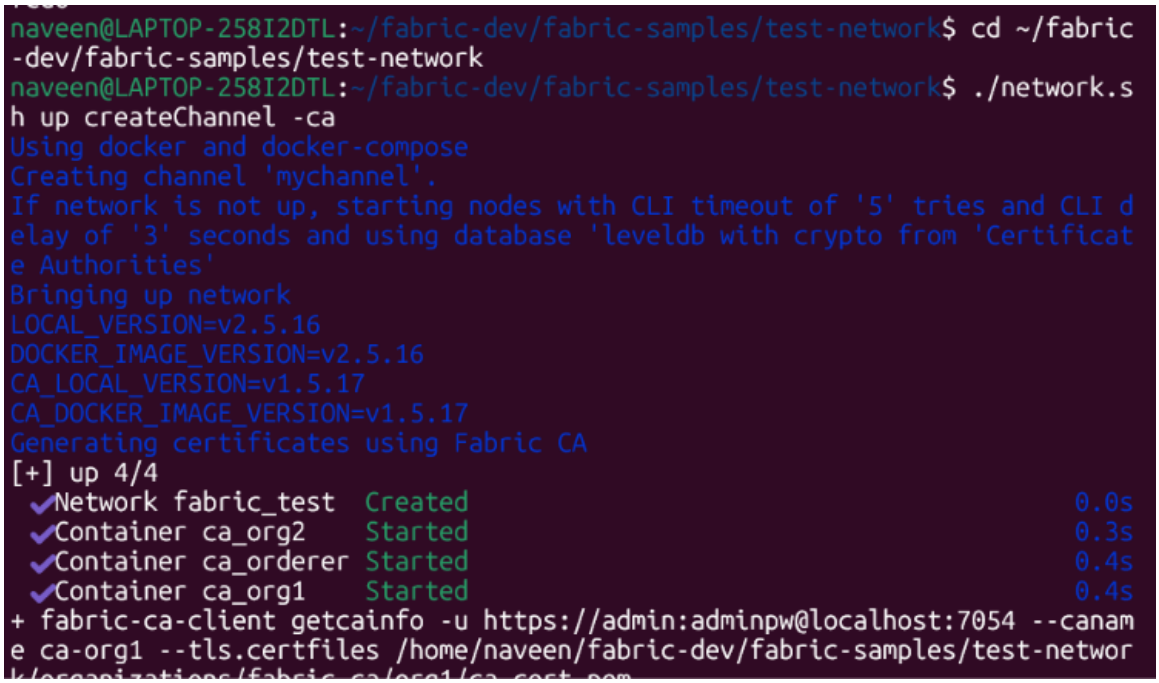
Navigate to the Hyperledger Fabric test network directory.

```
cd ~/fabric-dev/fabric-samples/test-network
```

Step 2: Start the Fabric Network

Start the Fabric network, create the application channel, and enable Certificate Authorities.

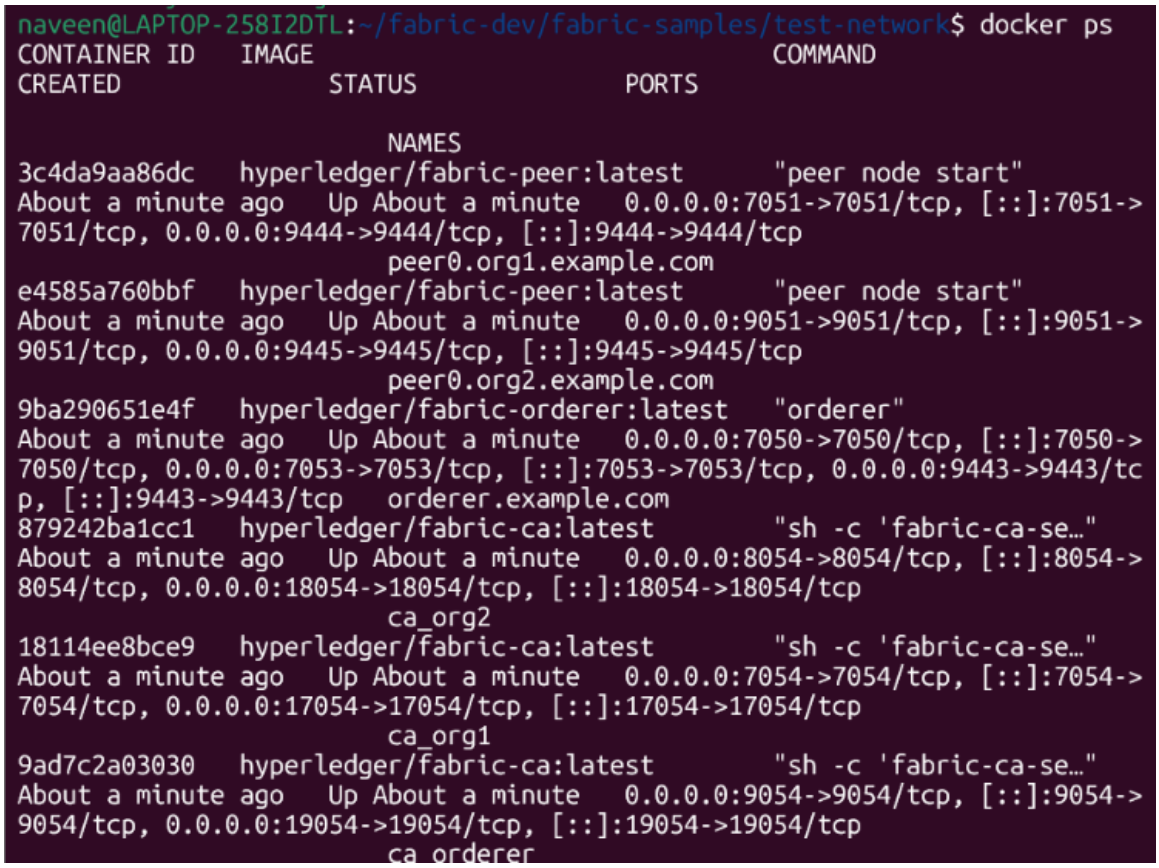
```
./network.sh up createChannel -ca
```



Step 3: Verify Running Docker Containers

Display all currently active Docker containers. This command can be used to verify that the peer, orderer, and Certificate Authority containers are running.

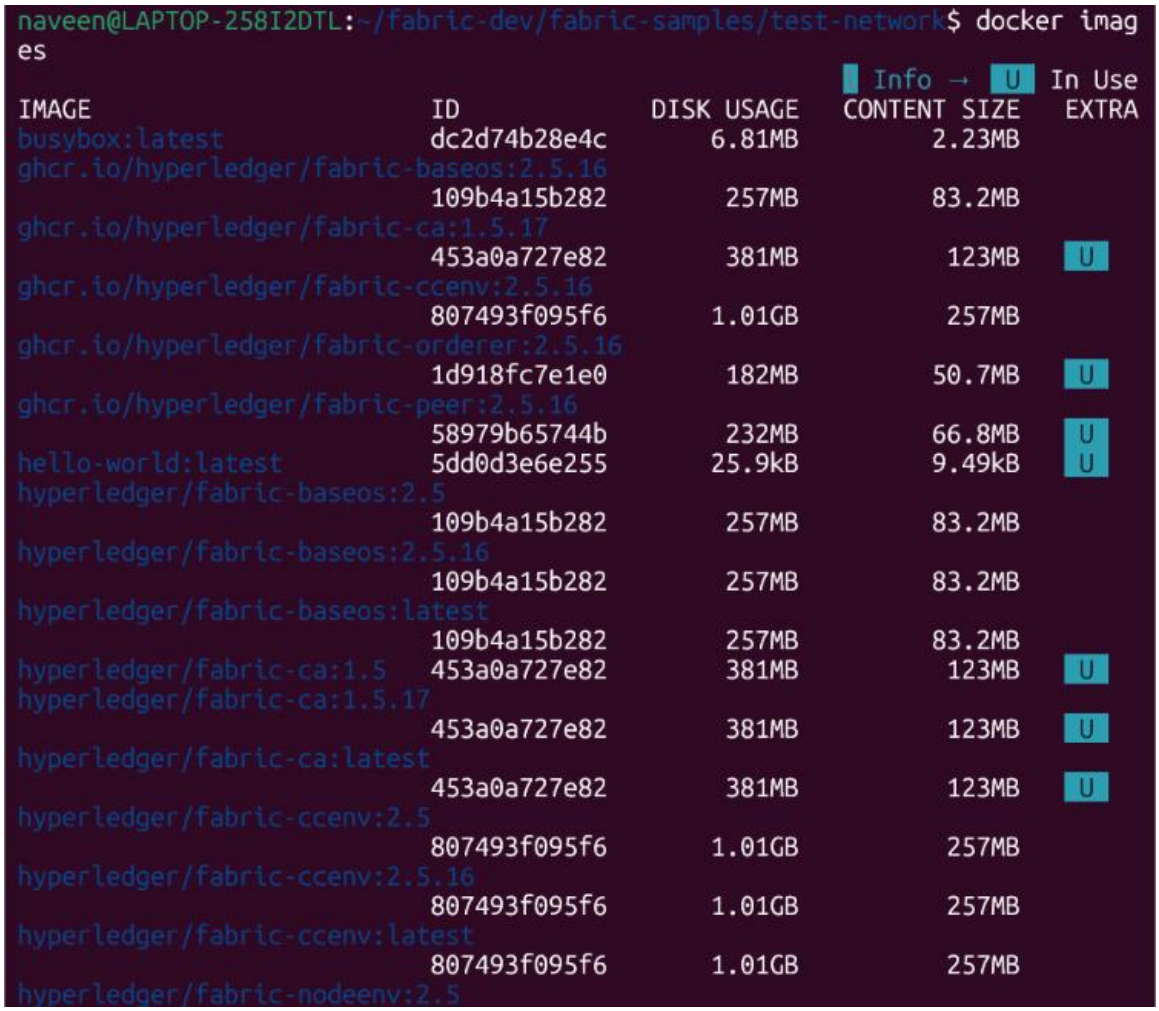
`docker ps`



Step 4: Display Docker Images

Display the Docker images available on the system. This allows the Fabric Docker images used by the network to be verified.

```
docker images
```



Step 5: Check Network Status

Display all Docker containers, including stopped containers. This helps monitor the current status of Fabric network containers.

docker ps -a

```
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$ docker ps -
a
CONTAINER ID   IMAGE                                COMMAND
CREATED              STATUS                       PORTS

                                NAMES
3c4da9aa86dc   hyperledger/fabric-peer:latest       "peer node start"
3 minutes ago        Up 3 minutes                 0.0.0.0:7051->7051/tcp, [::]:70
51->7051/tcp, 0.0.0.0:9444->9444/tcp, [::]:9444->9444/tcp
                                peer0.org1.example.com
e4585a760bbf   hyperledger/fabric-peer:latest       "peer node start"
3 minutes ago        Up 3 minutes                 0.0.0.0:9051->9051/tcp, [::]:90
51->9051/tcp, 0.0.0.0:9445->9445/tcp, [::]:9445->9445/tcp
                                peer0.org2.example.com
9ba290651e4f   hyperledger/fabric-orderer:latest    "orderer"
3 minutes ago        Up 3 minutes                 0.0.0.0:7050->7050/tcp, [::]:70
50->7050/tcp, 0.0.0.0:7053->7053/tcp, [::]:7053->7053/tcp, 0.0.0.0:9443->944
3/tcp, [::]:9443->9443/tcp   orderer.example.com
879242ba1cc1   hyperledger/fabric-ca:latest         "sh -c 'fabric-ca-se…"
3 minutes ago        Up 3 minutes                 0.0.0.0:8054->8054/tcp, [::]:80
54->8054/tcp, 0.0.0.0:18054->18054/tcp, [::]:18054->18054/tcp
                                ca_org2
18114ee8bce9   hyperledger/fabric-ca:latest         "sh -c 'fabric-ca-se…"
3 minutes ago        Up 3 minutes                 0.0.0.0:7054->7054/tcp, [::]:70
54->7054/tcp, 0.0.0.0:17054->17054/tcp, [::]:17054->17054/tcp
```

Step 6: Deploy Asset Transfer Chaincode

Deploy the JavaScript Asset Transfer chaincode to the Fabric network.

./network.sh deployCC -ccl javascript -ccn basic -ccp ../asset-transfer-basic/chaincode-javascript

```
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$ ./network.s
h deployCC -ccl javascript -ccn basic -ccp ../asset-transfer-basic/chaincode
-javascript
Using docker and docker-compose
deploying chaincode on channel 'mychannel'
executing with the following
- CHANNEL_NAME: mychannel
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
- CC_SEQUENCE: auto
- CC_END_POLICY: NA
- CC_COLL_CONFIG: NA
- CC_INIT_FCN: NA
- DELAY: 3
- MAX_RETRY: 5
- VERBOSE: false
executing with the following
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
+ '[' false = true ']'
+ peer lifecycle chaincode package basic.tar.gz --path ../asset-transfer-bas
ic/chaincode-javascript --lang node --label basic_1.0
+ res=0
Chaincode is packaged
Installing chaincode on peer0.org1...
Using organization 1
+ peer lifecycle chaincode queryinstalled --output json
+ grep '^basic_1.0:1d4d445573112813889f87c307bc2b5fbe664c87b24c035bee15cbcc7
f59b629$'
+ jq -r 'try (.installed_chaincodes[].package_id)'
```

Step 7: Verify Installed Chaincode

Query the installed chaincode packages. This command verifies that the chaincode package has been installed on the peer.

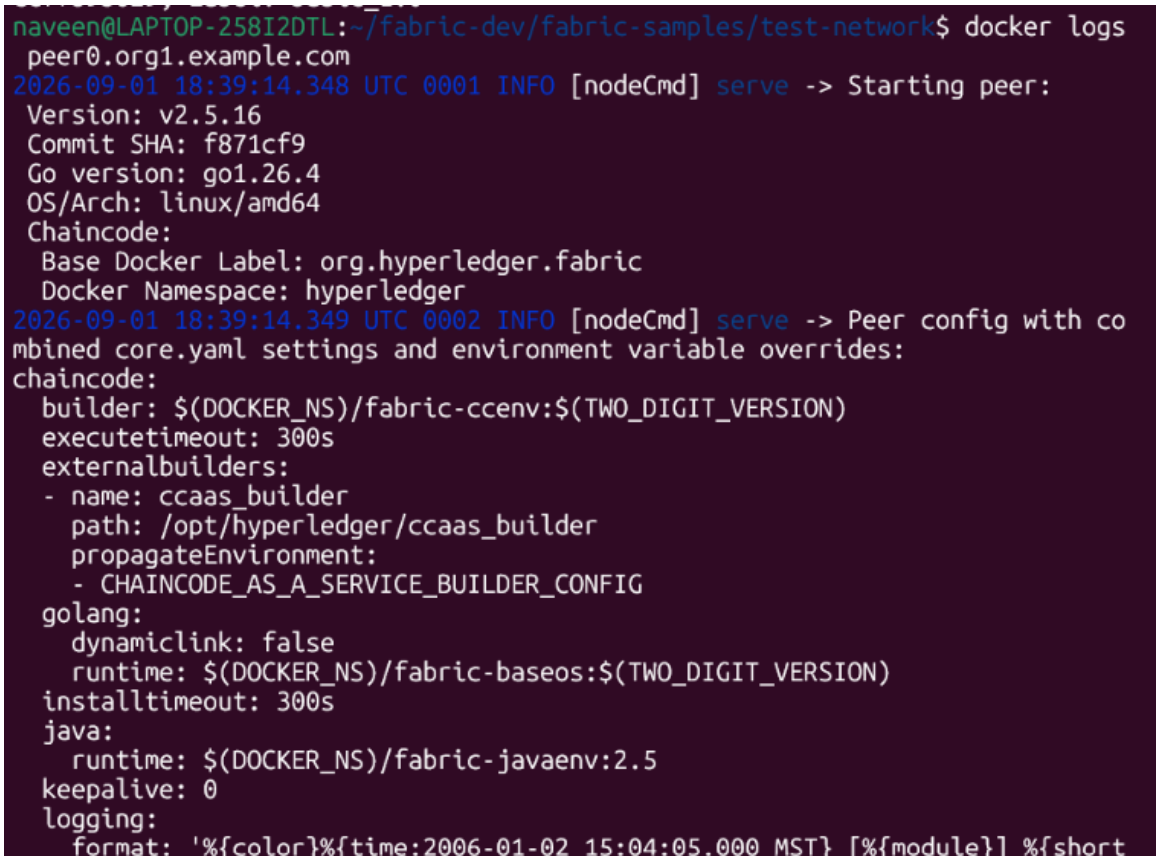
peer lifecycle chaincode queryinstalled

```
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$ peer lifecy
cle chaincode queryinstalled
Installed chaincodes on peer:
Package ID: basic_1.0:1d4d445573112813889f87c307bc2b5fbe664c87b24c035bee15cb
cc7f59b629, Label: basic_1.0
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$
```

Step 8: Monitor Peer Logs

Display the logs generated by the Org1 peer node. The logs can be used to monitor peer startup, connections, transactions, and chaincode-related activities.

docker logs peer0.org1.example.com



Step 9: Monitor Orderer Logs

Display the logs generated by the orderer node. The logs can be used to monitor the ordering service and transaction processing.

docker logs orderer.example.com

```
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$ docker logs
 orderer.example.com
2026-09-01 18:39:14.492 UTC 0001 INFO [localconfig] completeInitialization -
> Kafka.Version unset, setting to 0.10.2.0
2026-09-01 18:39:14.492 UTC 0002 INFO [orderer.common.server] prettyPrintStr
uct -> Orderer config values:
        General.ListenAddress = "0.0.0.0"
        General.ListenPort = 7050
        General.TLS.Enabled = true
        General.TLS.PrivateKey = "/var/hyperledger/orderer/tls/server.key"
        General.TLS.Certificate = "/var/hyperledger/orderer/tls/server.crt"
        General.TLS.RootCAs = [/var/hyperledger/orderer/tls/ca.crt]
        General.TLS.ClientAuthRequired = false
        General.TLS.ClientRootCAs = []
        General.TLS.TLSHandshakeTimeShift = 0s
        General.Cluster.ListenAddress = ""
        General.Cluster.ListenPort = 0
        General.Cluster.ServerCertificate = ""
        General.Cluster.ServerPrivateKey = ""
        General.Cluster.ClientCertificate = "/var/hyperledger/orderer/tls/se
rver.crt"
        General.Cluster.ClientPrivateKey = "/var/hyperledger/orderer/tls/ser
ver.key"
        General.Cluster.RootCAs = [/var/hyperledger/orderer/tls/ca.crt]
        General.Cluster.DialTimeout = 5s
        General.Cluster.RPCTimeout = 7s
        General.Cluster.ReplicationBufferSize = 20971520
        General.Cluster.ReplicationPullTimeout = 5s
        General.Cluster.ReplicationRetryTimeout = 5s
        General.Cluster.ReplicationBackgroundRefreshInterval = 5m0s
        General.Cluster.ReplicationMaxRetries = 12
        General.Cluster.SendBufferSize = 100
        General.Cluster.CertExpirationWarningThreshold = 168h0m0s
        General.Cluster.TLSHandshakeTimeShift = 0s
```

Step 10: Verify the Channel

List the channels joined by the peer. This verifies that mychannel has been successfully created and joined by the peer.

peer channel list

```
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$ peer channe
l list
2026-09-01 18:48:53.296 UTC 0001 INFO [channelCmd] InitCmdFactory -> Endorse
r and orderer connections initialized
Channels peers has joined:
mychannel
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$
```

Step 11: Stop the Fabric Network

Stop and remove the Fabric network. This removes the Fabric containers, network, and generated network artifacts.

```
./network.sh down
```

```
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$ ./network.s
h down
Using docker and docker-compose
Stopping network
[+] down 10/10
 ✔Container orderer.example.com          Removed                          0.6s
 ✔Container ca_org2                      Removed                          3.6s
 ✔Container ca_orderer                   Removed                          3.4s
 ✔Container ca_org1                      Removed                          3.7s
 ✔Container peer0.org1.example.com       Removed                          1.1s
 ✔Container peer0.org2.example.com       Removed                          1.2s
 ✔Network fabric_test                    Removed                          0.2s
 ✔Volume compose_peer0.org1.example.com  Removed                          0.0s
 ✔Volume compose_peer0.org2.example.com  Removed                          0.0s
 ✔Volume compose_orderer.example.com     Removed                          0.0s
Error response from daemon: get docker_orderer.example.com: no such volume
Error response from daemon: get docker_peer0.org1.example.com: no such volum
e
Error response from daemon: get docker_peer0.org2.example.com: no such volum
e
Removing remaining containers
Removing generated chaincode docker images
Untagged: dev-peer0.org2.example.com-basic_1.0-1d4d445573112813889f87c307bc2
b5fbe664c87b24c035bee15cbcc7f59b629-c68f8ecc2a0ff0cc1497615f5e33a8bfa908e0fd
3555f864cfa1398b52d341cb:latest
Deleted: sha256:f1510d88fcba098e575834af1608e83ef63e48f900a36ce993eddaa5bcad
16dd
Untagged: dev-peer0.org1.example.com-basic_1.0-1d4d445573112813889f87c307bc2
b5fbe664c87b24c035bee15cbcc7f59b629-d3cf9b1f4b9747cc8433ac0e74bfaeade1cf0b97
f5c1163835f604aa885d60dc:latest
Deleted: sha256:44df08245fcac3503020a8ed8dc047667f58d9e836d3de7093f55715fea1
df52
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$
```

Step 12: Clean Docker Resources

Optionally remove unused Docker containers, networks, images, and other resources. This command helps reclaim unused Docker resources and disk space.

docker system prune -f

```
naveen@LAPTOP-258I2DTL:~/fabric-dev/fabric-samples/test-network$ docker syst
em prune -f
Deleted Containers:
5a28b04fe92ab90d1a71785118fb0e7d69632d3c32969d14871a83c825821f10
3aa5cc9ef5c2c041327ac082380ea672b3ac6bb7f0eef915936ed7e84bd41ec3
7e032e35684240d1ce9cd9576ada1ffd19a5bb57486357f09aa6277232e4f090
3b6cde636b9561b3fe1aa9859e8a82ab5236fa86d21e4bc1e53e39a1102fea91
4d23498f291f2bd41bbc7ef972e4f8c32521f6a6df7face9425daefe5c4f6685
35600b781d32fe02b67344ec76cd46e1b3c10d15bb16f688cac6f18295087392
bb149863e3643a429a790f3b2b6803c7f31548718d7aaba748b60a51db13a8a1
261e8e0b9a5e0f5bbd4d164770f48ca5269ba52fff0bd286c6a268c8f07077ae
60885e44b10f78a42f2d5beb4db702e70f08d04be40a3ac796e93886c96b0d8a
1bf259c4c9f21b61d96ac932235c9841e3ebba12aa5612aa589004e76c79fd54
788dd2d9edc7867071e2846ca70b527a296c0c01e2dacf965da6770cdc4f7bd5
4f46225d08c8063cb2d2510067be2ad41eb4019344b4f16dbecc19b201f0b281
39aff2d0164fb917c07c52045ff7f0fcd9962a5390ccbb2d0632d2bceb500173
9dcd84f847ba7703bdf2a89352bffb0a0d7f72d4d36dcb7b2138665f7736bd0d
88a650ee3084ec1204b8ff6af6af6c23e14074b3f2b0e6232250a18ba36e8e5f
74550be244d07d752896a46e7d3369ba8734f9780eb10b42e44e41e8e29c38ff
16204bcea781b88ee63d733d141621bc2bb8d99d1c53a543a97b538f10a05318
aaa936e67b9c2214333fbc185d691281f81ccbf648905c9e3e582ccc29f5e460
0160d4746a4895c04ad915f143ecd7deb0d2726417beacaa2b47efa5b63b026c
3f8d72318fa14a45f880aad28882fe6cae467ff14b34f41b24942a6f0300aae7
472f1df41e071c1ecf66c8afec49437e5f7f6774a6c09ab3d2f189c2940dcb96
9c67cbdbe5eb1f86d64995f3494c85a223cb6c5a3e04c4df98262f25249542a0
ef6b3fc94047c325f4b4b81fc421eb12c58c68e211fbedbf96d11104e405f899
8984f615206da66bff8cf3948a883cc90baa2d8d534bb0ce008524a811f0276c
ba3f562e11333c871432629b31eef2b9618d2bd72e6d5270abca6c8f92987f92
7cf51d37101ad8cc7215579981b9c78026a18e92949b218cd15d2eebdfe9f19e
```

NETWORK MANAGEMENT WORKFLOW

Install Dependencies → Start Fabric Network → Create Channel → Verify Docker Containers → Deploy Chaincode → Verify Installed Chaincode → Monitor Peer Logs → Monitor Orderer Logs → Verify Channel → Stop Network → Clean Docker Resources

RESULT

The Hyperledger Fabric network was successfully deployed and managed. The peer nodes, ordering service, Certificate Authorities, Docker containers, and application channel were verified. JavaScript chaincode was deployed and its installation was verified. Peer and orderer logs were monitored, and the network was successfully stopped and cleaned up.